

ParaWeb: Parallel Programming Patterns for Web Development

Suejb Memeti

*Department of Computer Science, Blekinge Institute of Technology,
Karlskrona, Sweden.

Corresponding author(s). E-mail(s): suejb.memeti@bth.se;

Abstract

Modern web applications increasingly require computationally intensive processing, yet JavaScript, the dominant language of the web, has traditionally been limited to a single-threaded execution model. Node.js Worker Threads and browser Web Workers provide low-level mechanisms for parallel execution, but developers lack high-level abstractions that capture recurring parallel structures as reusable patterns. In this paper, we present ParaWeb, a TypeScript library that implements ten parallel programming patterns for server-side Node.js, client-side browser environments, and WebGPU compute shaders. ParaWeb provides three implementation variants for each pattern: a message-passing (MP) variant based on structured cloning via `postMessage`, a shared-buffer (Shared) variant that uses `SharedArrayBuffer` with typed array views, and a GPU variant that uses WebGPU compute shaders for hardware-accelerated execution. We describe the architecture, design decisions, and pattern-specific implementation strategies, and we evaluate the performance of all thirty implementations across three data sizes. Experimental evaluation results show that the CPU-based variants achieve speedups of up to 11.6x with 16 threads for compute-bound patterns, while the GPU variants reach speedups of up to 260x for compute-bound patterns with high arithmetic intensity such as Farm, Scatter, Reduce, and Map. A case study on five image-convolution filters further shows that GPU acceleration reaches up to 414x speedup over single-threaded CPU on non-separable kernels, with consistent scaling across 1024^2 , 2048^2 , and 4K images.

Keywords: Parallel programming, High-level abstractions, Parallel patterns, Web development, Node.js, Web Workers, WebGPU

1 Introduction

Modern web applications increasingly handle workloads that were traditionally reserved for desktop or server-side high-performance computing environments. Tasks such as image processing, real-time data analytics, scientific visualization, and machine learning inference [20] now routinely execute within web browsers and Node.js server processes. However, JavaScript, the primary programming language of web development, was originally designed around a single-threaded event loop model, which limits its ability to exploit the multi-core CPUs and GPUs that are common in today’s hardware. While this model simplifies concurrency reasoning and avoids many classes of synchronization errors, it leaves significant computational resources unused when applications encounter CPU-bound workloads.

The introduction of Worker Threads in Node.js¹ and Web Workers in browser environments² has made true parallelism possible within the JavaScript ecosystem. These mechanisms allow developers to spawn separate execution contexts that run concurrently on different CPU cores. Supporting both server-side and client-side parallelism is important for several reasons: it avoids overloading servers by offloading computation to client devices, and it allows privacy-sensitive workloads, such as medical image processing or personal data transformations, to remain on the client when appropriate. However, these low-level primitives require developers to manually manage thread creation, data partitioning, message passing, synchronization, and result aggregation, which is a non-trivial and error-prone programming task.

The concept of *algorithmic skeletons*, originally introduced by Cole [2], provides a structured approach to expressing parallelism through high-level patterns such as Map, Reduce, Pipeline, and Farm. Several mature frameworks in the C++ ecosystem, such as SkePU [3], FastFlow [7], and GrPPI [10], have shown that pattern-based parallel programming improves programmer productivity while providing portable performance across different hardware platforms. However, to the best of our knowledge, no pattern-based parallel programming library exists for the JavaScript ecosystem that covers Node.js Worker Threads, browser Web Workers, and WebGPU compute shaders together with multiple implementation strategies. The closest peer is River Trail [13], which was an engine-level effort and has been discontinued.

In this paper, we present ParaWeb³, a TypeScript library that implements ten parallel programming patterns for web development. The patterns are organized into three categories: (1) data-parallel patterns, including Map, Filter, Reduce, Scan, and Scatter; (2) task-parallel patterns, including Farm, Pipeline, and Divide-and-Conquer; and (3) specialized patterns, including MapReduce and Stencil. For each pattern, we provide three implementation variants: a message-passing (MP) variant that distributes data via structured cloning through `postMessage`, a shared-buffer (Shared) variant that uses `SharedArrayBuffer` with `Float64Array` views to allow zero-copy data sharing between threads, and a GPU variant that offloads computation to the graphics processor using WebGPU compute shaders written in WGSL (WebGPU Shading Language). All three variants run in modern browsers without engine changes. WebGPU is

¹<https://nodejs.org/api/worker-threads.html>

²<https://www.w3.org/TR/workers/>

³Source code: <https://github.com/sm13294/paraweb-js>

supported in Chrome 113+, Edge 113+, Firefox 121+, and Safari 18+, and the Shared variant additionally requires cross-origin isolation headers. The design of ParaWeb is inspired primarily by the GrPPI framework [10], adapted for the JavaScript ecosystem.

Major contributions of this paper include:

- A library of ten parallel programming patterns under a single typed API, with three implementation variants per pattern (message-passing, shared-buffer, GPU) covering Node.js Worker Threads, browser Web Workers, and WebGPU compute shaders.
- An empirical evaluation of all thirty implementations across three input sizes and four parallel thread counts plus a sequential baseline, on Node.js and in the browser, with the same per-element workload on every variant.
- A browser-based case study on five image-convolution filters (Gaussian blur, box blur, sharpen, emboss, edge detection) showing GPU acceleration scaling consistently across 1024^2 , 2048^2 , and 4K images.

The rest of this paper is structured as follows. Section 2 reviews related work on parallel pattern frameworks and JavaScript parallelism. Section 3 presents the architecture and design decisions of ParaWeb. Section 4 describes pattern-specific implementation details. Section 5 outlines the experimental setup. Section 6 reports and discusses performance results, including the workload-dependent variant-selection observations and current limitations. Section 7 concludes the paper and outlines future work.

2 Related Work

ParaWeb builds on prior research in two areas: algorithmic skeletons and pattern-based parallel programming libraries in established HPC ecosystems, and parallel and GPU-accelerated execution models for the web. In this section, we review each of these areas and position ParaWeb with respect to existing approaches.

2.1 Algorithmic Skeletons and Parallel Pattern Libraries

The idea of expressing parallelism through reusable high-level patterns has been developed for several decades. Asanović et al. introduced the “Berkeley Dwarfs,” a taxonomy of thirteen algorithmic motifs representing the dominant computational patterns in parallel applications [1]. Cole [2] introduced *algorithmic skeletons* as a structured way of expressing parallelism through high-level patterns such as Map, Reduce, Pipeline, and Farm. Both approaches capture recurring parallel structures as reusable abstractions, reducing programmer burden while allowing portable and efficient parallel execution.

Several C++ frameworks have since shown the effectiveness of implementing parallel patterns as reusable constructs. SkePU [3] provides Map, Reduce, MapReduce, Scan, and Stencil patterns with portable execution across CPUs, GPUs, and HPC clusters, using OpenMP, CUDA, and OpenCL backends, with type-safe skeleton programming added in recent versions. Muesli [4] is another C++ skeleton library with OpenMP, CUDA, and MPI backends, providing both data- and task-parallel skeletons.

FastFlow [7] provides streaming and task-parallel patterns such as Pipeline, Farm, and Divide-and-Conquer, using lock-free queues for high throughput. Kokkos [5] addresses performance portability across many-core architectures through polymorphic memory access patterns and parallel execution abstractions such as `parallel_for` and `parallel_reduce`. Intel TBB [8] and OpenMP [9] include parallel patterns implicitly through generic algorithms and compiler directives.

GrPPI (Generic Reusable Parallel Patterns Interface) [10] provides a single interface for patterns such as Map, Reduce, MapReduce, Stencil, Pipeline, Farm, Filter, and Divide-and-Conquer, targeting different backends including OpenMP, Intel TBB, C++ threads, and FastFlow by selecting an execution policy at compile time. GrPPI has been evaluated for stream processing [11] and multi-core performance [12], showing that a small, well-chosen set of patterns is sufficient for a broad range of parallel workloads, and serves as the primary design inspiration for ParaWeb.

2.2 Parallel and GPU Computing in Web Environments

Parallel computing on the web has historically been limited by JavaScript’s single-threaded execution model. The closest peer to ParaWeb is River Trail [13], which extended JavaScript with a `ParallelArray` type supporting Map, Reduce, Scan, Filter, and Scatter, executing on multi-core CPUs and GPUs via OpenCL. River Trail required engine-level modifications to SpiderMonkey and was tied to OpenCL as its GPU backend. Mozilla’s complementary Parallel JS (PJs) effort [14] took a similar engine-level approach. Both projects were ultimately abandoned (River Trail’s code was removed from Firefox in 2015) because shipping a parallel JavaScript runtime required every engine vendor to adopt the same engine-level changes, which did not happen, and OpenCL itself never gained traction on the web. Web Workers provide a standardized, low-level mechanism for parallel task execution in browser environments, but they lack structured abstractions; developers must manually manage message passing, data partitioning, and synchronization. Their performance scalability has been characterised in detail across browsers and CPU configurations [22], and earlier work has investigated unified distributed-and-parallel JavaScript programming models built on top of them [25]. Several libraries, such as `parallel.js`⁴ and `workerpool`⁵, expose limited forms of Map or SIMD-like processing, but none offer a full pattern-based abstraction covering the range of parallel patterns found in established frameworks. Node.js Cluster⁶ enables process-based parallelism but focuses on request-level load balancing rather than structured data or task parallelism.

For compute-intensive workloads, two complementary paths have emerged for speeding up code on the web. WebAssembly [6] defines a portable, low-level byte-code that browsers execute at near-native CPU speed. Recent research shows that WebAssembly performance is typically within $\sim 2\times$ of native code on standard benchmarks, with the remaining gap attributed to register pressure and memory-management overheads [18]. WebAssembly speeds up single-thread execution and addresses a different concern than ParaWeb’s Worker-Thread parallelism, so the two

⁴<https://github.com/parallel-js/parallel.js>

⁵<https://github.com/josdejong/workerpool>

⁶<https://nodejs.org/api/cluster.html>

combine naturally by running WebAssembly-compiled functions inside the worker pool. WebGPU [19] is a more recent W3C standard that provides low-level access to GPU compute and rendering capabilities from JavaScript. Unlike its predecessor WebGL, which was designed for graphics rendering, WebGPU includes a compute shader pipeline that allows general-purpose GPU (GPGPU) computations using the WGSL shading language. WebGPU is available in Node.js through packages such as `webgpu` (Dawn-based). Recent work has begun characterising its performance for general-purpose scientific computing in the browser [24]. GPU.js [21] is an earlier JavaScript library that transpiles JavaScript functions to WebGL shader code for GPU execution, but it offers only Map-like operations and does not support the full range of parallel patterns.

In contrast, ParaWeb is a pure user-space library built on standardized APIs (Node.js Worker Threads, browser Web Workers, and WebGPU compute shaders), and therefore deploys to any modern runtime today without engine changes. It also covers a broader pattern set than River Trail, adding Stencil, MapReduce, and the task-parallel Farm, Pipeline, and Divide-and-Conquer patterns, as well as stream stage operators (filter, window-reduce, iterate) inside Pipeline.

3 Design

This section describes the architecture of ParaWeb and the design decisions behind its three implementation variants. We first present the overall architecture and the rationale for restricting input data to numeric arrays, then classify the ten supported patterns by their computational characteristics, and finally describe the trade-offs between the message-passing, shared-buffer, and GPU execution backends.

3.1 Architecture Overview

ParaWeb follows a modular architecture in which each parallel pattern is implemented as a separate, self-contained class. This design promotes code reuse, simplifies testing and maintenance, and allows individual patterns to evolve independently. Despite their independence, all patterns share a common set of design principles that ensure consistency across the library.

Each pattern exposes a unified interface with an optional `numThreads` parameter that allows developers to control the degree of parallelism. When the argument is omitted, ParaWeb defaults to the number of available processing cores, queried via `os.cpus().length` in Node.js or `navigator.hardwareConcurrency` in the browser, so applications use all available cores without manual configuration. The patterns handle all Worker Thread management details, including thread creation, task distribution, result collection, and resource cleanup. TypeScript provides compile-time type checking, so that type errors are caught during development rather than at run-time. Error handling ensures that worker failures are properly managed, with resource cleanup even under error conditions.

3.2 Data Type Restriction

ParaWeb restricts input data to numeric arrays. The MP variants support one-dimensional and two-dimensional numeric arrays, while the Shared variants operate on one-dimensional numeric arrays backed by `Float64Array` buffers. This design decision is motivated by several considerations. First, it simplifies the implementation by removing complex type detection and polymorphic handling logic. Second, it improves performance by avoiding type checking overhead on the parallel execution path. Third, it aligns with established frameworks such as GrPPI [10] and SkePU [3], which similarly focus on structured numeric data. Fourth, the JavaScript engine can make stronger assumptions about data layout and memory access patterns when operating on typed arrays, which allows for better optimization.

This restriction is justified by the observation that most computationally intensive parallel workloads operate on numeric data. Application domains such as image processing, signal analysis, scientific computation, financial modeling, and data transformation pipelines mostly work with numeric arrays. By focusing on this common case, ParaWeb achieves simplicity and performance without sacrificing expressiveness for the workloads that benefit most from parallelization.

3.3 Pattern Classification

The ten patterns implemented in ParaWeb can be classified into three categories based on their computational characteristics. Data-parallel patterns, including Map, Filter, Reduce, Scan, and Scatter, apply a uniform operation across a collection of elements. Their inter-element dependencies are either absent (Map, Filter) or structurally regular (associative in Reduce and Scan, index-driven in Scatter), which admits a partition-compute-combine parallel decomposition. Task-parallel patterns, including Farm, Pipeline, and Divide-and-Conquer, focus on organizing work into tasks that can be distributed across workers, with varying degrees of dependency and coordination between tasks. Specialized patterns, including Stencil and MapReduce, address specific computational structures that arise frequently in practice but require specialized handling, such as neighborhood-based computation or combined transformation and aggregation.

Figure 1 provides abstract views of each pattern, illustrating the data flow from inputs to outputs. Detailed descriptions of how each pattern is implemented in ParaWeb are presented later in Section 4.

3.4 Three Implementation Variants

For each pattern, ParaWeb provides three implementation variants that explore different points in the parallelism trade-off space: a message-passing (MP) variant prioritizing simplicity and broad data-type support at the cost of serialization overhead; a shared-buffer (Shared) variant that eliminates copying for numeric data at the cost of explicit coordination and a typed-array restriction; and a GPU variant offering the highest throughput for compute-bound data-parallel patterns but requiring WGSL operations and `f32` precision. Section 4 describes each variant in detail.

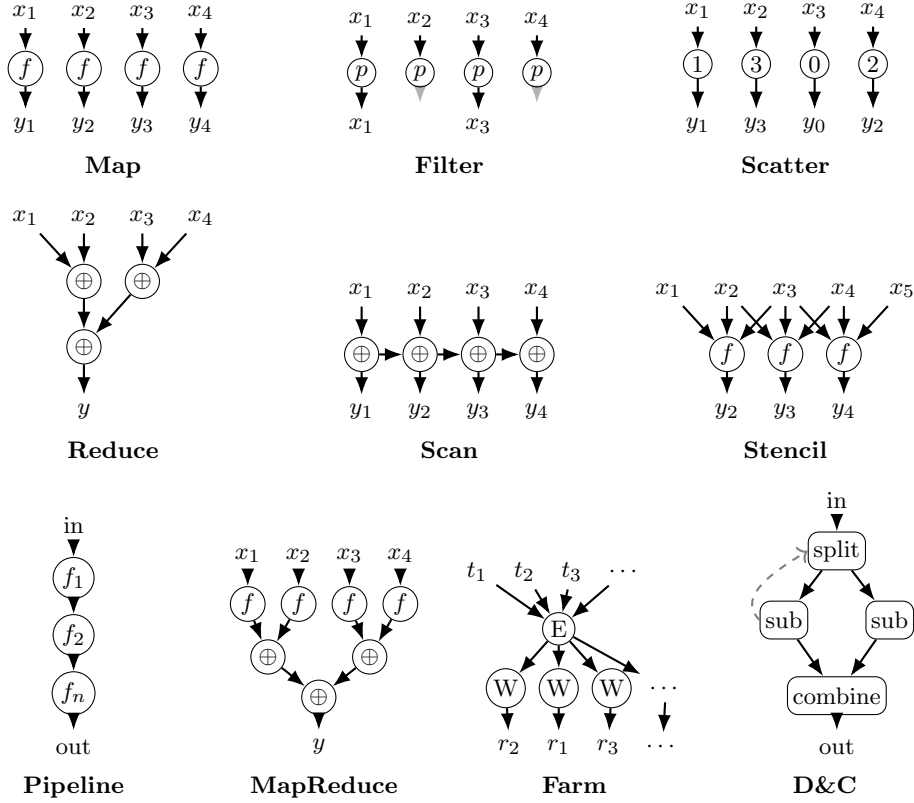


Fig. 1: Abstract views of the ten ParaWeb patterns.

3.5 Programming Interface

ParaWeb exposes each pattern as a free function under a single `paraweb` namespace, so users write parallel code that looks like ordinary sequential JavaScript. An equivalent Map implementation using raw Node.js Worker Threads typically spans roughly 40 lines across two files: a main script that chunks the input, spawns workers, wires `message`, `error`, and `exit` handlers, awaits all promises, and concatenates partial results; and a separate worker script that rebuilds the user function from its source string and posts the per-chunk result. With ParaWeb the same workload is one line. Listing 1 lists the public interface of all ten patterns, and the three variants (MP, Shared, GPU) are reached through `paraweb.mp.*`, `paraweb.shared.*`, and `paraweb.gpu.*`; the top-level names (`paraweb.map`, `paraweb.reduce`, ...) select the empirically best variant per pattern (Section 6).

```

1 import paraweb from "paraweb";
2
3 // Data-parallel
4 paraweb.map(fn, input, threads?);
5 paraweb.filter(pred, input, threads?);
6 paraweb.reduce(op, input, identity, threads?);
7 paraweb.scan(op, input, identity, threads?);
8 paraweb.scatter(input, indices, default?, conflictFn?, threads?);

```

```

9
10 // Task-parallel
11 paraweb.farm(fn, input, threads?);
12 paraweb.pipeline(stages, input, threads?);
13 paraweb.divideAndConquer(divideFn, conquerFn, baseFn, input, threads?);
14
15 // Specialized
16 paraweb.stencil(fn, input, window, threads?, edgeOption?);
17 paraweb.mapReduce(mapFn, reduceOp, input, threads?);

```

Listing 1: Public interface of the ten ParaWeb patterns. Each call returns a **Promise**; **threads** defaults to the number of available CPU cores.

The three variants share the same calling convention, so switching between them is a single property access. Listing 2 shows the same Map across MP, Shared, and GPU. The GPU variant accepts a WGSL expression (or a built-in operation name) instead of a JavaScript function, because the body executes on the device. The call site is otherwise identical.

```

1 import paraweb from "paraweb";
2 const f = (x: number) => Math.sin(x) * Math.cos(x * 0.5);
3 const y1 = await paraweb.mp.map(f, input); // Worker Threads, postMessage
4 const y2 = await paraweb.shared.map(f, input); // SharedArrayBuffer, zero-copy
5 const y3 = await paraweb.gpu.map( // WebGPU compute shader
6   { wgsl: "sin(x) * cos(x * 0.5)" }, input);

```

Listing 2: Switching between the three implementation variants.

Composition of patterns is expressed through ordinary JavaScript control flow. Listing 3 builds a three-stage processing pipeline (denoise, feature extraction, classification). Since every pattern returns a **Promise**, patterns compose naturally with **await** and can be mixed with non-parallel code without any special syntax.

```

1 import paraweb from "paraweb";
2 const denoise = (x: number) => 0.25 * x + 0.5 * x + 0.25 * x;
3 const features = (x: number) => Math.tanh(x);
4 const classify = (x: number) => x > 0.5 ? 1 : 0;
5 const labels = await paraweb.pipeline(
6   [denoise, features, classify], pixels, /* threads */ 16);

```

Listing 3: Three-stage image-processing pipeline.

4 Implementation

This section describes the implementation of ParaWeb, including the worker thread management infrastructure shared across patterns, the function serialization and communication protocol used between the main thread and workers, the implementation of the three execution variants (MP, Shared, GPU), and the pattern-specific implementation details for each of the ten patterns.

4.1 Worker Thread Management

All worker-based patterns in ParaWeb use a reusable worker pool (**WorkerPool**) as the underlying mechanism for parallel execution, using Node.js Worker Threads on the server and Web Workers in the browser. Workers are lazily created, cached by script path and pool size, and reused across invocations to amortize creation overhead,

an approach broadly consistent with prior work on dynamic Web Worker pool management for JavaScript applications [23]. The input is partitioned into $\lceil n/p \rceil$ -element chunks, where n is the array length and p the number of threads. The thread count p is capped at $\min(p, n)$ to avoid spawning empty workers on small inputs.

4.2 Function Serialization and Communication

Worker Threads and Web Workers operate in separate memory spaces, which means that functions cannot be passed directly between threads. ParaWeb addresses this through function serialization: user-supplied functions are converted to strings using `Function.toString()` and reconstructed in worker threads using `new Function()`, which compiles the function in the worker’s global scope. Note that `new Function()` does not preserve the surrounding lexical scope, so the user-supplied function must be self-contained: references to external helper functions or to captured variables from the enclosing scope are not resolved in the worker and will throw at execution time. In practice this means user code either inlines the helper logic inside the function body, or passes additional data as part of the input array rather than as a closed-over variable. Built-in globals such as `Math`, `Array`, and typed-array constructors are available in the worker and can be used normally. Task messages in MP variants contain the serialized function code, input data, and pattern-specific parameters, while Shared variants carry buffer references, index ranges, and coordination metadata such as atomic counters. Workers report errors using an `"error"` string protocol and shut down upon receiving a `"terminate"` message.

4.3 MP and Shared CPU Variants

The MP variants partition input arrays into chunks and transfer data to workers using `postMessage()`, which relies on the structured clone algorithm and incurs copying overhead proportional to input size. The Shared variants place input and output data in `SharedArrayBuffer` instances backed by `Float64Array` views, allowing workers to read and write shared memory regions without copying. The generic `divideAndConquer` API still delegates to MP-style transfer, since arbitrary user-supplied subproblems are not guaranteed to be array-shaped. The specialised Shared D&C paths used in our evaluation (`fft` and `sort`) operate directly on shared buffers, with workers dispatched per Cooley-Tukey stage. While shared buffers eliminate transfer overhead for large arrays, they introduce synchronization and coordination costs. For instance, the Shared Filter variant uses a two-pass approach (count matching elements, then write results), and the Shared Farm variant uses an atomic counter to distribute work dynamically. Our experimental results show that the Shared variant outperforms MP for most patterns on large datasets.

4.4 GPU Variants via WebGPU

The GPU variants offload computation to the graphics processor using WebGPU compute shaders. They accept operations specified as WGSL expressions or predefined built-in operation names, such as `"square"`, `"collatz_steps"`, or `"is_prime"`, which are injected into a compute shader template. The GPU infrastructure consists

of four core modules: (i) a singleton GPU device manager that acquires and caches the `GPUDevice`, (ii) buffer utilities for uploading `Float32Array` data to GPU storage buffers and reading back results, (iii) a pipeline cache that avoids shader recompilation across repeated invocations, and (iv) a shader builder that generates WGSL compute shaders from operation specifications. Note that WebGPU compute shaders only support 32-bit floating-point arithmetic (`f32`), so GPU results have lower numerical precision than the `Float64Array`-based CPU variants. This trade-off is acceptable for workloads where throughput matters more than precision, such as image processing, scientific visualization, and data analytics.

For data-parallel patterns such as Map, Filter, and Stencil, each input element maps directly to a GPU thread (invocation), with a workgroup size of 256 threads. The GPU hardware scheduler distributes workgroups across compute units, providing automatic load balancing. For Reduce, we use a hierarchical reduction scheme with workgroup-level parallel reduction using shared memory, followed by iterative passes until a single value remains. Filter uses a three-phase stream compaction approach: a mark phase that evaluates the predicate, a prefix-sum phase that computes output indices, and a compact phase that writes matching elements to the output buffer. For task-parallel patterns such as Farm and Pipeline, the GPU variants delegate to the Map kernel, since the GPU’s hardware scheduler provides work distribution analogous to the CPU Farm pattern.

4.5 Pattern-Specific Implementations

The five *data-parallel* patterns all start by chunking the input into p aligned slices. Map and Filter apply the user function or predicate independently per chunk and concatenate the results, with Filter’s Shared variant using a two-pass count-then-write approach to handle variable-length output. Scatter redistributes values to destinations given by an index array, with each worker producing index-value pairs and the main thread assembling an output of length $\max(\text{index}) + 1$, resolving collisions with a last-write policy or optional combiner (matching River Trail’s scatter semantics [13]). Reduce aggregates each chunk to a partial result in parallel and combines the partials on the main thread using the same associative operator, with the user-supplied `identity` value as the initial accumulator. Scan (inclusive prefix-scan) uses a two-pass parallel algorithm: each worker computes a local inclusive scan and reports its chunk total, the main thread computes an exclusive scan of chunk totals, and each worker then adds its prefix offset to every element in its chunk. Reduce and Scan both require an associative binary operation, with $O(n/p + p)$ complexity.

The three *task-parallel* patterns add coordination on top of the worker pool. Farm distributes variable-cost tasks across workers using one of two strategies depending on the variant. The MP variant uses static chunking (p contiguous ranges, one per worker), since per-task message passing would dominate cost. The Shared variant uses a lock-free atomic counter so that workers self-schedule by pulling batches of $\lceil n/p \rceil$ indices via `Atomics.add` on a shared counter, avoiding both per-task messages and main-thread coordination. Pipeline processes data through sequential stages with intra-stage data parallelism, where each stage is either a plain function (applied element-wise as a parallel map) or one of three stream stage operators inspired by

GrPPI’s streaming patterns [10]: *stream filter* (drops items failing a predicate), *stream window-reduce* (collapses each sliding window into one value), and *stream iterate* (applies a fixed-point loop per item). Divide-and-Conquer (D&C) recursively decomposes problems through a user-supplied `divideFn`, distributes top-level subproblems across threads, and combines results via `conquerFn`. The base-case threshold trades more parallelism for more task-management overhead.

The two *specialized* patterns handle recurring computational structures. Stencil applies neighborhood-based computation with overlap regions of $\lfloor s/2 \rfloor$ elements around each worker’s slice, so boundary computations resolve without inter-worker communication. Configurable edge handling makes it suitable for image convolution, signal processing, and finite-difference methods. MapReduce composes a parallel Map phase with a subsequent Reduce phase by reusing both implementations, mirroring the computational structure of Hadoop and Spark adapted for shared-memory worker threads.

5 Experimental Setup

To evaluate the performance of ParaWeb across different workloads, data sizes, and thread configurations, we designed a set of benchmarks that combine representative data sizes with computationally intensive per-pattern algorithms. In this section, we describe the test configurations and the specific algorithms used to exercise each pattern.

5.1 Test Configurations

We evaluate each pattern across three data sizes of increasing scale: Medium (100K elements), Large (1M elements), and Extremely Large (5M–10M elements). For each data size, we test a plain-JavaScript sequential baseline plus four parallel configurations of 2, 4, 8, and 16 threads. These size configurations correspond to representative use cases: data transformation pipelines (100K), scientific computation (1M), and large-scale data analytics (5M–10M).

Each configuration is executed for five runs after two warmup runs, and we report the mean execution time. Speedup is the ratio of a plain-JavaScript sequential baseline (main thread, no workers, no `SharedArrayBuffer`) to the parallel time, using identical inputs and the same per-element workload as the parallel run.

5.2 Test Algorithms

Each pattern is evaluated using a computationally intensive algorithm designed to produce sufficient per-element work to amortize parallelization overhead:

- **Map:** A 50-iteration iterative transform per element ($\sin$, $\cos$, $\sqrt{}$, and a polynomial step), providing a compute-bound workload with no inter-element dependencies.
- **Filter:** A 30-iteration trigonometric predicate followed by a threshold comparison, which produces data-dependent output size with a roughly 50% pass rate on the chosen input distribution.

- **Reduce:** Associative sum over a 10-iteration sin/cos per-element transform (roughly 1/5 the arithmetic intensity of Map/MapReduce). The transform is supplied as a separate `mapFn` argument so the combine operator stays associative.
- **Scan:** Inclusive prefix sum (associative) fused with the 50-iteration trigonometric transform per element, so that the two-pass prefix-sum algorithm is exercised on a compute-bound workload with enough per-element work to amortize the phase-2 offset pass.
- **Scatter:** Index remapping with randomly generated destination indices in $[0, n/4)$, applying the 50-iteration trigonometric transform per element (matching Map’s arithmetic intensity) and summing values at conflicting positions on CPU.
- **Stencil:** Weighted five-point neighborhood sum (weights $[1, 2, 3, 2, 1]$) followed by a 15-iteration trigonometric refinement per element, producing high arithmetic intensity per stencil application.
- **Farm:** Per-task workload over starting numbers in $[1000, 2000)$ combining a Collatz step-count (variable cost, ~ 50 –250 steps) with a 200-iteration trigonometric refinement. The Collatz prefix preserves Farm’s variable-cost characteristic while the refinement raises per-task cost into the millisecond range so coordination amortizes.
- **Divide-and-Conquer:** Parallel Cooley-Tukey radix-2 FFT [15] on a complex array of N points.
- **Pipeline:** Two stages, a 30-iteration trigonometric transform followed by a 20-iteration polynomial evaluation with rounding, testing sequential stage composition with intra-stage parallelism.
- **MapReduce:** The 50-iteration trigonometric transform used by Map, followed by associative sum aggregation, testing the fused map-then-reduce workflow.

The same per-element transforms are fused into the WGSL kernels for Reduce, Scan, and Scatter so that GPU and CPU measurements in Table 2 compare equivalent workloads.

6 Experimental Evaluation

In this section, we evaluate ParaWeb across all ten patterns, covering both CPU variants (MP and Shared) and the GPU variant, in Node.js and in the browser. We first describe the experimental environment, then report CPU scaling for data-parallel patterns and for task-parallel and specialized patterns, followed by browser-based results, GPU acceleration results against the corresponding CPU baseline in each runtime, and a real-time image-convolution case study.

6.1 Experimental Environment

Experiments were conducted on an Apple MacBook Pro with the M3 Max chip (16 CPU cores: 12 performance + 4 efficiency; 48 GB unified memory; integrated 40-core Apple GPU), running macOS 26.4, Node.js v22.19.0, and TypeScript v5.3.3. GPU benchmarks were executed both under Node.js (using the Dawn-based `webgpu` npm package) and under headless Chromium (using the native WebGPU implementation over Apple Metal). Each pattern was executed with numeric arrays of the specified

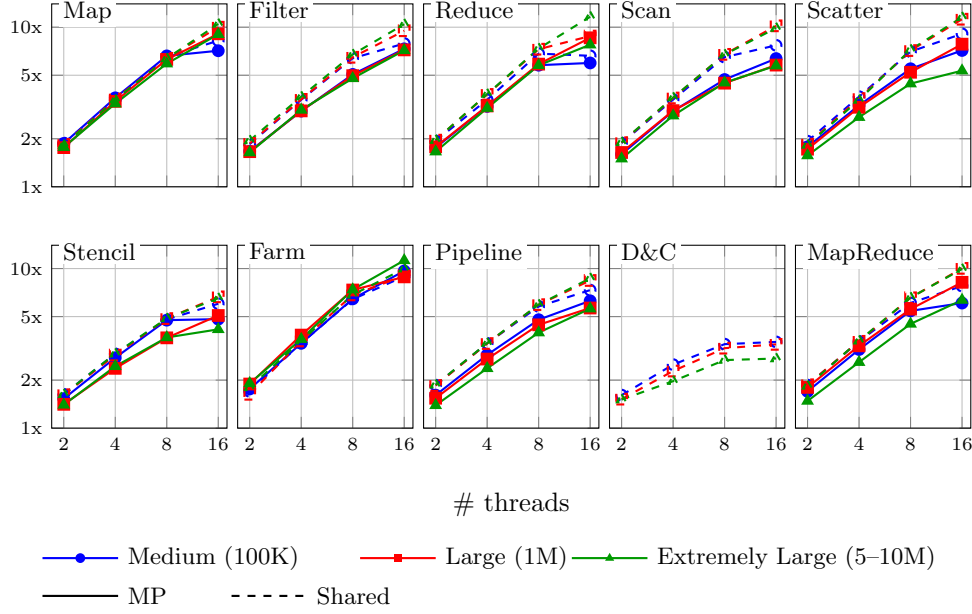


Fig. 2: Speedup relative to a plain-JavaScript sequential baseline (main thread, no workers, no SharedArrayBuffer) for all ten patterns across data sizes and thread counts (2–16). The y-axis is $\log_{10}$ -scaled; the 1x gridline marks the sequential baseline. Lines below 1x indicate that the parallel variant is slower than the sequential loop.

sizes, measuring wall-clock execution time. Note that all measurements include two warmup runs to minimize the impact of JIT compilation and cache effects. Results are verified for correctness by comparing outputs across different thread counts to ensure that parallelization does not alter computational results. We additionally collected browser-based results using Web Worker implementations against the same plain-JavaScript sequential baseline with thread counts of 2, 4, 8, and 16, reported at Extremely Large inputs in Table 1.

6.2 Data-Parallel Pattern Results

Figure 2 reports CPU speedup curves for all ten patterns. At 16 threads, Map, Filter, and Scatter achieve comparable speedups of 10.48x, 10.44x, and 11.54x in the Shared variant, and 9.03x, 7.16x, and 5.34x in the MP variant, which indicates that the compute-bound per-element transform amortizes worker-coordination overhead for both variants. Filter’s good Shared performance is notable given that variable-length output requires a two-pass count-then-write approach. The savings from avoiding data copying outweigh the synchronization cost. Scatter Shared has the highest 16T speedup of any CPU pattern by writing slice-locally to a shared output buffer and resolving index collisions through a lock-free atomic compare-exchange on a per-output last-writer index, whereas Scatter MP pays for serializing per-worker output chunks back to the main thread.

Reduce and Scan exhibit an arithmetic-intensity threshold: both patterns have a synchronizing combine phase the parallel variants must traverse twice, and below a certain per-element compute cost the coordination overhead dominates. With a per-element trigonometric transform fused into the operator (10 iterations for Reduce, 50 for Scan, matching Map’s arithmetic intensity), per-element compute amortizes the coordination cost: Reduce reaches 11.62x (Shared) and 7.77x (MP), and Scan reaches 9.88x and 5.74x respectively at 16 threads. Without that transform, pure associative sum and pure prefix-sum both fall below 1x at every thread count, because the reduction machinery (or for Scan, the second-pass memory traffic) outweighs the trivial per-element work. Therefore, parallel reduction is counterproductive on memory-bandwidth-bound workloads with insufficient per-element compute.

6.3 Task-Parallel and Specialized Pattern Results

Farm reaches 11.25x (MP) and 9.99x (Shared) at 16 threads on Node.js, and 11.11x (MP) and 11.85x (Shared) in the browser, on Extremely Large inputs (5M elements; capped below 10M because the per-task workload is heavier than the data-parallel benchmarks). Both variants partition the input into p contiguous ranges via static chunking. The Shared variant additionally pays one `Atomics.add` per range to support dynamic load balancing. On the Collatz-plus-trigonometric workload, per-task cost variance is mild and static chunking is already balanced, so the dynamic dispatch is unused work, and the two variants converge to within $\sim 10\%$ of each other in both runtimes. The Shared variant’s distinguishing capability translates into measurable benefit only when per-task cost variance is high enough that static chunking creates stragglers, e.g., when heavy tasks cluster in the input array.

On 8M complex points, the Shared D&C variant reaches 2.73x at 16 threads on Node.js and 2.62x in the browser. The CPU ceiling is intrinsic to FFT: each of the $\log_2 N \approx 23$ butterfly stages must complete before the next, costing a per-stage coordination round-trip that even Shared cannot avoid. MP cannot scale on FFT at all: per-stage cross-worker data exchange is required, and structured-clone pays for it on every round-trip. As a result, Node MP measures 0.23x at 2 threads (a 4.3x slowdown over sequential) and recovers only to 0.57x at 16 threads, never reaching parity with the sequential baseline. Note that we omit the MP D&C curves from Figure 2 for clarity, since their sub-1x trajectories would compress the readable y-axis range without adding analytical content.

Pipeline, Stencil, and MapReduce all scale predictably with Shared outperforming MP: 8.69x vs. 5.61x, 6.52x vs. 4.16x, and 9.93x vs. 6.35x respectively at 16 threads on Extremely Large inputs. Pipeline benefits from Shared’s elimination of inter-stage copying. Stencil’s 15-iteration trigonometric refinement per element amortizes the overlap regions duplicated across chunk boundaries. MapReduce mirrors Map’s profile because the map phase dominates and the reduce phase adds only a small sequential cost. Within Pipeline, the stream stage operators reach stream-iterate 9.63x, window-reduce 4.12x, and a composed map+filter+window-reduce 5.35x at 16T on 10M-element inputs.

6.4 Browser-Based Results

To evaluate ParaWeb in the browser, we re-ran the identical workloads in headless Chromium using the Web Worker variants, served with cross-origin isolation (COOP/COEP) headers to enable `SharedArrayBuffer`. Table 1 reports speedups against the same plain-JS sequential baseline used in the Node.js evaluation, at Extremely Large inputs (10M elements; Farm at 5M). Browser speedups follow the Node.js ranking closely: Farm Shared reaches 11.85x at 16 threads, Scatter Shared 11.06x, Farm MP 11.11x, Scan Shared 10.23x, Map Shared 10.02x, MapReduce Shared 9.87x, Filter Shared 9.71x, Reduce Shared 9.43x, Pipeline Shared 8.01x, and Stencil Shared 6.10x. D&C reaches 2.62x (Shared) and 2.11x (MP) on the FFT workload, limited by the per-stage synchronization shared by all variants.

Table 1: Web Worker speedup (relative to a plain-JavaScript sequential baseline: main thread, no workers, no `SharedArrayBuffer`) for Extremely Large inputs (10M elements; Farm at 5M to match its heavier per-task workload, D&C at 8M to keep N a power of two for the radix-2 FFT), measured in headless Chromium. Bold values mark the per-pattern peak; italic values below 1.0x indicate the parallel variant is slower than the sequential loop.

T	Variant	Map	Filter	Reduce	Scan	Scatter	Stencil	Farm	Pipeline	D&C	MapReduce
2	MP	1.71	1.53	1.12	1.35	1.44	1.14	1.85	1.39	1.11	1.60
	Shared	1.89	1.87	1.93	1.87	1.91	1.38	1.85	1.81	1.53	1.86
4	MP	3.05	2.64	1.84	2.36	2.46	1.88	3.62	2.37	1.97	2.89
	Shared	3.58	3.53	3.74	3.52	3.50	2.52	3.66	3.29	2.15	3.51
8	MP	5.30	4.25	2.79	3.84	3.63	2.92	6.91	3.72	2.21	4.79
	Shared	6.66	6.33	6.88	6.72	6.86	4.35	6.98	5.64	2.57	6.49
16	MP	7.82	5.92	3.43	5.16	4.39	3.69	11.11	5.29	2.11	7.18
	Shared	10.02	9.71	9.43	10.23	11.06	6.10	11.85	8.01	2.62	9.87

6.5 GPU Acceleration Results

We evaluate the GPU variants of all ten patterns in both runtimes, using the same data sizes and equivalent workloads as the CPU benchmarks, and compare GPU execution against the corresponding CPU variant within the same runtime: Worker-Thread CPU in Node.js and Web Worker CPU in the browser. Since the GPU handles parallelism internally through its hardware scheduler, there is no thread count parameter. Each benchmark measures the total execution time, including data upload, kernel execution, and result readback. Table 2 reports the Node.js and browser GPU results side by side.

The GPU outperforms 16-thread CPU on data- and task-parallel patterns with high per-element arithmetic intensity, with the largest Node.js speedups on Farm

Table 2: GPU vs. CPU performance on Extremely Large inputs (10M elements; Farm at 5M and D&C at 8M to match CPU caps) for both runtimes. Speedups are reported against the plain-JS sequential baseline (“vs seq”) and against the best parallel CPU variant at 16 threads (“vs 16T”). Bold marks the peak per column; italic values below 1.0x indicate CPU beats GPU. All rows compare equivalent workloads: Reduce, Scan, Scatter, Farm, and D&C GPU kernels apply the same per-element transform or algorithm as their CPU counterparts.

Pattern	Node.js			Browser		
	GPU (ms)	vs seq	vs 16T	GPU (ms)	vs seq	vs 16T
Map	163	63.5x	6.1x	176	54.7x	5.5x
Filter	950	8.0x	<i>0.8x</i>	969	6.5x	<i>0.7x</i>
Reduce	22	94.2x	8.1x	30	46.4x	4.9x
Scan	222	67.0x	6.8x	191	65.2x	6.4x
Scatter	85	239.6x	20.8x	114	113.5x	10.3x
Stencil	160	17.7x	2.7x	164	16.4x	2.7x
Farm	105	260.5x	22.9x	108	222.6x	21.9x
Pipeline	324	25.7x	3.0x	324	20.6x	2.6x
D&C	479	3.6x	1.3x	423	3.0x	1.1x
MapReduce	193	53.7x	5.4x	191	49.8x	5.1x

(260.5x), Scatter (239.6x), Reduce (94.2x), Scan (67.0x), Map (63.5x), MapReduce (53.7x), Pipeline (25.7x), and Stencil (17.7x), with speedups of 3–23x over the best 16-thread CPU configuration for most patterns. The browser column mirrors this ranking with lower ratios because both CPU and GPU share the Apple Metal driver through Chromium’s scheduler. Filter loses to CPU-16T in both runtimes because V8 JIT-optimizes its CPU path well [26] and the three-phase mark/scan/compact GPU kernel is dominated by the sequential single-workgroup prefix-sum in compaction. D&C reaches a GPU speedup of 3.6x in Node.js and 3.0x in the browser, which is bounded by FFT’s inherent cross-stage synchronization: the $\log_2 N$ stages must execute in order even though the pairwise operations within a stage are independent.

6.6 Case Study: Real-Time Image Convolution

To show that ParaWeb is useful for a concrete web scenario, we present a case study of 2D image convolution, which is a core building block in privacy-preserving browser-based applications such as photo editors, medical imaging viewers, and real-time video filters. In such applications, user data remains on the client device and computation must complete within the typical interactive latency budget of approximately 100 ms per operation for slider-based filtering to feel responsive. We evaluate five common convolution filters: Gaussian blur, box blur, unsharp-mask sharpen, directional emboss, and Difference-of-Gaussians edge detection [17].

Each filter maps onto one or two of ParaWeb’s patterns. Gaussian blur and box blur decompose into two 1D Stencil passes (horizontal followed by vertical), exploiting kernel separability to reduce per-pixel work from $O(r^2)$ to $O(r)$. Emboss uses a single 2D Stencil with a full $(2r + 1) \times (2r + 1)$ non-separable kernel. Unsharp-mask sharpen

composes a Stencil (blur) with a Map that combines source and blurred pixels element-wise ($out = src + (src - blur)$). DoG edge detection composes two Stencils at different radii with a Map that subtracts and biases. Sharpen and DoG are therefore naturally expressed as short Pipelines. Our case-study implementation uses a specialized worker on top of ParaWeb’s worker pool and `SharedArrayBuffer` utilities that follows this same structure, and the GPU path is realized as one or two fused WGSL compute shaders (16×16 workgroups) per filter so intermediate buffers stay on the device. Both variants are measured in the steady state of an interactive session: the CPU path reuses its worker pool and shared input/output buffers across calls, and the GPU path caches its compiled pipeline, device buffers, and bind group. Per-call work is the image upload plus compute. Figure 3 plots speedup relative to the sequential CPU baseline as the kernel radius grows from 1 to 20 on a 4K image, on a log y-axis.

The separable filters (Gaussian, box) and the composite unsharp-mask sharpen share the same scaling profile: GPU speedup climbs from around 5x at radius 1 to 40–50x at radius 20, while CPU-Shared 16T plateaus around 8–10x because memory bandwidth rather than compute dominates. The non-separable emboss filter, whose cost scales as $(2r + 1)^2$ per pixel, places significantly higher demand on the parallel infrastructure: the GPU reaches a 414x speedup at radius 20 (75 ms versus 31 s sequential) because its 5,120 parallel lanes (40 Apple GPU cores $\times$ 128 ALUs) absorb the quadratic work that saturates even the 16-thread CPU configuration at 10x. This exceeds the GPU speedups in Table 2 for the natively transcendental-heavy patterns (e.g., 260x on Farm, 240x on Scatter, 64x on Map) because emboss at radius 20 is 1681 fused multiply-adds per pixel with no transcendentals, matching the GPU’s FMA lanes almost perfectly, whereas the pattern microbenchmarks are intentionally transcendental-heavy and thus bound by the GPU’s special-function units rather than its FMA throughput. Edge detection via DoG, which is two separable blurs plus a per-pixel combine, falls between these regimes and reaches 85x on the GPU. Note that the same sweep on 1024^2 and 2048^2 images yields the same ranking and almost-identical GPU-to-CPU ratios, with the emboss GPU peak at radius 20 staying in the 380–414x band across all three resolutions. This confirms that the scaling shape is a property of the algorithms rather than of a specific image size. This case study demonstrates that for compute-bound browser workloads such as client-side image and video processing, ParaWeb shifts applications from batch-style processing with visible latency to genuinely interactive behavior without sending user data to a server.

6.7 Discussion

Across configurations, we observe several trends. First, on Node.js the Shared variant outperforms MP on most compute-bound patterns, reaching 9–12x at 16 threads (Figure 2). Farm is the one exception, where Shared (9.99x) trails MP (11.25x) because static chunking already balances the workload and the atomic counter adds unused coordination cost. Second, the variant ordering is not stable across runtimes: in the browser the ranking flips for Farm, with Shared (11.85x) overtaking MP (11.11x) (see Table 1). Third, Reduce and Scan are memory-bandwidth bound with a pure-associative workload, so parallel scaling is limited or negative when no per-element

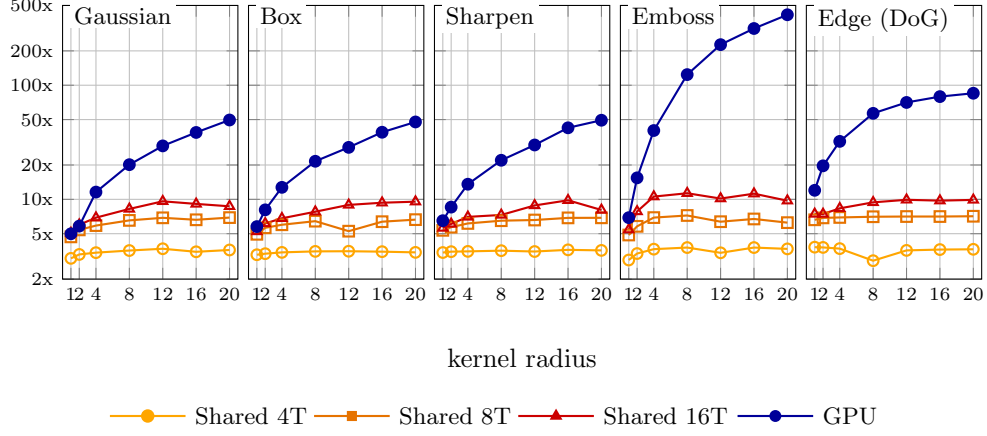


Fig. 3: Image-convolution case study (3840x2160): speedup relative to the sequential CPU baseline as the kernel radius grows, for each of the five filters. Y-axis is $\log_{10}$ -scaled.

compute is added. Both patterns scale past 9x once a per-element transform is fused into the operator.

These results indicate that variant defaults are workload-dependent rather than fixed per pattern. Reduce and Scan illustrate this in particular: both fall below 1x without a per-element transform but scale past 9x once one is fused into the operator, so the appropriate variant depends on arithmetic intensity rather than on the pattern alone. D&C illustrates a complementary case: it plateaus at $\sim 2.7x$ because Cooley-Tukey FFT’s cross-stage synchronization is intrinsic to the algorithm rather than a property of any one variant, and the appropriate strategy for FFT-shaped workloads is therefore the GPU rather than a different CPU variant. ParaWeb’s static default selection (Shared for most patterns on CPU, with GPU recommended for compute-bound patterns with high arithmetic intensity such as Farm, Scatter, Map, MapReduce, Pipeline, and Stencil) is therefore a useful starting point, while a workload-aware selector is a longer-term direction.

The three variants reflect complementary trade-offs. MP is simpler but incurs copying. Shared eliminates copying but adds coordination complexity and restricts data to typed arrays. GPU offers the highest throughput for high-arithmetic-intensity patterns but requires WGSL and `f32` precision. Current limitations include cross-origin isolation for browser Shared variants, `f32` GPU precision, a sequential single-workgroup prefix-sum inside GPU Filter’s compaction step (the standalone Scan pattern already uses a parallel Blelloch kernel [16]), the requirement that user-supplied functions be self-contained because `new Function()` in the worker does not preserve the caller’s lexical scope, and static variant selection.

7 Conclusion and Future Work

In this paper, we presented ParaWeb, a TypeScript library that implements ten parallel programming patterns for Node.js, browsers, and WebGPU, with three implementation variants per pattern. CPU variants reach speedups of up to 11.6x in Node.js at 16 threads on compute-bound patterns. GPU variants reach 260x (Farm) on Node.js for patterns with high arithmetic intensity. Our image convolution case study reaches 414x GPU speedup on the non-separable emboss filter at radius 20. Future work may focus on dynamic runtime variant selection, fused GPU kernels for composed patterns, and heterogeneous workload distribution that splits a single pattern invocation across CPU workers and the GPU concurrently rather than picking one. A complementary direction is a cloud-continuum-aware scheduler that decides per call whether to execute on client resources, offload to a server runtime, or split work between the two based on input size, network conditions, and data residency constraints.

References

- [1] Asanović, K., et al.: The landscape of parallel computing research: A view from Berkeley. Technical Report UCB/EECS-2006-183, University of California, Berkeley (2006)
- [2] Cole, M.: Algorithmic Skeletons: Structured Management of Parallel Computation. MIT Press, Cambridge, MA (1989)
- [3] Ernstsson, A., Ahlqvist, J., Zouzoula, S., Kessler, C.W.: SkePU 3: Portable high-level programming of heterogeneous systems and HPC clusters. *Int. J. Parallel Program.* **49**(6), 846–866 (2021)
- [4] Ciechanowicz, P., Kuchen, H.: Enhancing Muesli’s data parallel skeletons for multi-core computer architectures. In: *Proc. IEEE Int. Conf. on High Performance Computing and Communications (HPCC)*, pp. 108–113 (2010)
- [5] Edwards, H.C., Trott, C.R., Sunderland, D.: Kokkos: Enabling manycore performance portability through polymorphic memory access patterns. *J. Parallel Distrib. Comput.* **74**(12), 3202–3216 (2014)
- [6] Haas, A., Rossberg, A., Schuff, D.L., Titzer, B.L., Holman, M., Gohman, D., Wagner, L., Zakai, A., Bastien, J.: Bringing the web up to speed with WebAssembly. In: *Proc. ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI)*, pp. 185–200 (2017)
- [7] Aldinucci, M., Danelutto, M., Kilpatrick, P., Torquati, M.: FastFlow: High-level and efficient streaming on multicore. In: *Programming Multi-core and Many-core Computing Systems*, pp. 261–280. Wiley (2017)
- [8] Reinders, J.: Intel Threading Building Blocks: Outfitting C++ for Multi-core Processor Parallelism. O’Reilly Media (2007)
- [9] OpenMP Architecture Review Board: OpenMP application programming interface, version 5.2 (2021). <https://www.openmp.org/specifications/>
- [10] del Rio Astorga, D., Dolz, M.F., Fernández, J., García, J.D.: A generic parallel pattern interface for stream and data processing. *Concurrency and Computation: Practice and Experience* **29**(24), e4175 (2017)

- [11] del Rio Astorga, D., Dolz, M.F., Fernández, J., García, J.D.: An auto-tuning approach for parallel multi-pattern applications. In: Proc. ICPP Workshops, pp. 1–10 (2016)
- [12] Dolz, M.F., Fernández, J., García, J.D., del Rio Astorga, D.: Evaluating the performance of parallel programming patterns on multi-core systems. In: Proc. HiPC, pp. 1–12 (2017)
- [13] Herhut, S., Hudson, R.L., Shpeisman, T., Sreeram, J.: River Trail: A path to parallelism in JavaScript. In: Proc. OOPSLA, pp. 729–744 (2013)
- [14] Matsakis, N.D.: Parallel closures: a new twist on an old idea. In: Proc. 4th USENIX Workshop on Hot Topics in Parallelism (HotPar ’12), Berkeley, CA (2012)
- [15] Cooley, J.W., Tukey, J.W.: An algorithm for the machine calculation of complex Fourier series. *Math. Comput.* **19**(90), 297–301 (1965)
- [16] Blleloch, G.E.: Prefix sums and their applications. In: Reif, J.H. (ed.) *Synthesis of Parallel Algorithms*, pp. 35–60. Morgan Kaufmann (1990)
- [17] Marr, D., Hildreth, E.: Theory of edge detection. *Proc. R. Soc. Lond. B* **207**(1167), 187–217 (1980)
- [18] Jangda, A., Powers, B., Berger, E.D., Guha, A.: Not so fast: analyzing the performance of WebAssembly vs. native code. In: Proc. USENIX Annual Technical Conference (USENIX ATC), pp. 107–120 (2019)
- [19] W3C GPU for the Web Working Group: WebGPU. W3C Candidate Recommendation Draft, 10 March 2026. <https://www.w3.org/TR/webgpu/>
- [20] Smilkov, D., Thorat, N., Assogba, Y., Yuan, A., Kreeger, N., Yu, P., Zhang, K., Cai, S., Nielsen, E., Soergel, D., Bileschi, S., Terry, M., Nicholson, C., Gupta, S.N., Sirajuddin, S., Sculley, D., Monga, R., Corrado, G., Viégas, F.B., Wattenberg, M.: TensorFlow.js: Machine Learning for the Web and Beyond. In: Proc. 2nd Conference on Systems and Machine Learning (SysML/MLSys), arXiv:1901.05350 (2019)
- [21] Sapuan, F., Saw, M., Cheah, E.: General-purpose computation on GPUs in the browser using gpu.js. *Computing in Science & Engineering* **20**(1), 33–42 (2018)
- [22] Verdú, J., Pajuelo, A.: Performance scalability analysis of JavaScript applications with Web Workers. *IEEE Computer Architecture Letters* **15**(2), 105–108 (2016)
- [23] Verdú, J., Costa, J.J., Pajuelo, A.: Dynamic web worker pool management for highly parallel JavaScript web applications. *Concurrency and Computation: Practice and Experience* **28**(13), 3525–3539 (2016)
- [24] Aldahir, A.: Evaluation of the performance of WebGPU in a cluster of web-browsers for scientific computing. Dissertation, Umeå University (2022). <https://urn.kb.se/resolve?urn=urn:nbn:se:umu:diva-197058>
- [25] Welc, A., et al.: Generic workers: Towards a unified distributed and parallel JavaScript programming model. In: Proc. Workshop on Programming Support Innovations for Emerging Distributed Applications (PSI EtA). ACM (2010)
- [26] Martinsen, J.K., et al.: Combining thread-level speculation and just-in-time compilation in Google’s V8 JavaScript engine. *Concurrency and Computation: Practice and Experience* **29**(1), e3826 (2017)